

Vai trò của Coordinator và Router trong kiến trúc Coordinator Pattern

Trong **Coordinator Pattern**, logic điều hướng (navigation flow) của ứng dụng được tách ra khỏi các ViewController và đặt vào các **Coordinator**. Mục tiêu là phân chia trách nhiệm rõ ràng: ViewController chỉ lo hiển thị giao diện và tương tác UI, còn Coordinator chịu trách nhiệm **điều phối luồng màn hình**. Để hiện thực hoá kiến trúc này một cách nhất quán và dễ bảo trì, hai protocol chính thường được định nghĩa là **Coordinator** và **Router**. Mỗi protocol này giải quyết một vấn đề thiết kế cụ thể trong tổ chức luồng điều hướng của ứng dụng iOS, giúp cải thiện tính **phân tách trách nhiệm**, **dễ kiểm thử (testability)** và **dễ bảo trì (maintainability)**.

Coordinator protocol – Chuẩn hóa điều phối flow và quản lý lifecycle

Coordinator protocol định nghĩa giao diện chung cho tất cả các coordinator trong ứng dụng. Nói cách khác, nó chuẩn hóa những gì một coordinator **phải làm**. Thông thường protocol này bao gồm các yêu cầu như: phương thức khởi động luồng (`start()`), danh sách **child coordinators** để quản lý các luồng con, và có thể cả các cơ chế kết thúc luồng (ví dụ: callback khi flow hoàn thành). Việc có một **Coordinator** protocol thống nhất mang lại những lợi ích sau:

- **Chuẩn hóa và tập trung hóa logic điều hướng:** Mọi **Coordinator** cụ thể đều tuân theo cùng một hợp đồng, đảm bảo chúng có các phương thức cần thiết để điều phối màn hình. Điều này giúp tập trung hóa logic điều hướng ở mức coordinator thay vì rải rác trong nhiều ViewController. Kết quả là ViewController không còn phải biết chi tiết luồng chuyển màn hình, chúng **ủy quyền việc điều hướng cho coordinator**. Ví dụ, thay vì `LoginViewController` tự đẩy màn hình Home khi đăng nhập thành công, nó sẽ thông báo cho `LoginCoordinator` để coordinator quyết định chuyển sang **Home flow**.
- **Quản lý child coordinators và vòng đời luồng:** Thông qua thuộc tính `children` (danh sách các coordinator con) được định nghĩa trong protocol, một coordinator có thể giữ tham chiếu đến các **flow con** đang hoạt động. Điều này giải quyết vấn đề tổ chức luồng phức tạp: một flow lớn (parent) có thể khởi chạy flow con (child) và theo dõi vòng đời của nó. Khi flow con hoàn thành hoặc hủy, parent coordinator có thể giải phóng nó khỏi danh sách children, tránh rò rỉ bộ nhớ. Việc **decouple** (tách biệt) quan hệ parent-child thông qua protocol cho phép một coordinator cha quản lý nhiều coordinator con khác nhau một cách thống nhất ¹. Nói cách khác, parent coordinator chỉ biết các child của mình qua kiểu giao diện **Coordinator** chung, không phụ thuộc vào class cụ thể của child, giúp cấu trúc app linh hoạt và rõ ràng hơn.
- **Cấu trúc ứng dụng rõ ràng, dễ mở rộng:** Mô hình coordinator tạo ra một **sơ đồ luồng** rõ ràng cho ứng dụng. Mỗi chức năng lớn (ví dụ: luồng onboarding, luồng authentication, luồng chính của ứng dụng) có thể được quản lý bởi một coordinator riêng. Nhờ có **Coordinator** protocol, việc thêm

một luồng mới chỉ cần tạo một class coordinator mới implement protocol này, mà không ảnh hưởng tới các phần khác ². ViewController trở nên tái sử dụng hơn vì không gắn chặt với luồng cụ thể nào; để tạo flow mới, chỉ việc tạo coordinator mới và sử dụng lại các ViewController cần thiết trong flow đó. Điều này cải thiện **maintainability**: thay đổi hay mở rộng luồng ít tác động dây chuyền, do logic điều hướng mỗi luồng đã tách biệt.

- **Tách biệt navigation khỏi business logic**: Theo đúng tinh thần thiết kế, Coordinator chỉ điều phối màn hình và chuyển dữ liệu giữa các ViewController, **không xử lý logic nghiệp vụ cụ thể của tính năng** ³. Business logic vẫn thuộc về ViewModel/Interactor (nếu dùng MVVM, VIPER, v.v.), còn Coordinator chỉ quan tâm đến *trình tự* và *cách kết nối* các màn hình. Sự phân tách này tăng tính **single responsibility**: mỗi thành phần có vai trò riêng, giúp code dễ hiểu và dễ kiểm thử hơn.

Tóm lại, Coordinator protocol ra đời để giải quyết vấn đề **“Massive ViewController”** trong các kiến trúc cũ. Bằng cách buộc tất cả coordinator tuân theo một chuẩn, chúng ta có một cách tiếp cận thống nhất để điều phối luồng trong app. Mọi luồng đều có vòng đời rõ ràng (start → chạy flow → kết thúc), có thể lồng ghép nhiều flow (parent/child) mà không làm rối code của ViewController. Điều này làm giảm sự phụ thuộc trực tiếp giữa các ViewController (chúng không trực tiếp biết nhau hay biết luồng tổng thể), toàn bộ logic chuyển màn hình tập trung một tầng trên (**Coordinator**) ⁴, giúp **routing logic dễ đọc, dễ bảo trì và mở rộng hơn** ⁴.

Router protocol – Tách biệt cơ chế điều hướng UI khỏi logic luồng

Trong khi Coordinator lo *điều phối nên mở màn hình nào*, thì Router protocol chịu trách nhiệm *thực thi việc điều hướng UI cụ thể*. Nói cách khác, Router đóng vai trò tách biệt **“what”** và **“how”**: Coordinator quyết định *sẽ điều hướng đến đâu*, còn Router quyết định *chuyển màn hình bằng cách nào (push, present, pop, ...)*. Việc đưa vào Router protocol giải quyết những vấn đề sau:

- **Decouple Coordinator khỏi chi tiết UI (UIKit)**: Nếu không có router, một coordinator thường giữ tham chiếu trực tiếp đến UINavigationController hoặc UIWindow và gọi các phương thức UIKit (như pushViewController, present) để chuyển màn hình. Cách này hoạt động nhưng làm cho coordinator **phụ thuộc chặt vào UIKit**, gây khó khăn khi kiểm thử và tái sử dụng. Router protocol được thiết kế để khắc phục điều đó: Coordinator chỉ biết đến một interface trừu tượng để điều hướng, không cần biết chi tiết triển khai. Cụ thể, Router protocol định nghĩa các hành vi điều hướng cần có, ví dụ: methods present(...), dismiss(...) hoặc push/pop view controller. Tất cả **router cụ thể** (như router dùng UINavigationController, router dùng modal presentation, v.v.) đều phải implement các phương thức này ⁵. Nhờ đó, **Coordinator không cần gọi thẳng hàm UIKit**, mà chỉ việc gọi router (vốn đã được inject vào). Điều này **tách rời logic flow khỏi kỹ thuật điều hướng UI**, giúp coordinator **không phụ thuộc vào chi tiết triển khai điều hướng** ⁶.
- **Thay thế linh hoạt và tái sử dụng Router**: Với Router protocol, ta có thể có nhiều implement khác nhau của router và lựa chọn tùy tình huống. Ví dụ, có AppRouter quản lý root window, NavRouter quản lý push/pop trong UINavigationController, hoặc router đặc biệt cho chuyển cảnh tùy chỉnh. Coordinator có thể dùng bất kỳ đối tượng nào tuân thủ Router protocol, do đó ta dễ dàng thay thế cơ chế điều hướng mà **không cần thay đổi code coordinator**. Chẳng hạn, trong một số trường hợp trên iPad muốn present màn hình bằng popover thay vì push vào navigation stack – ta chỉ cần một router triển khai cách present đó, còn code điều phối trong coordinator giữ nguyên.

Sự linh hoạt này cũng hỗ trợ **mở rộng và bảo trì**: thay đổi hoạt ảnh chuyển cảnh hay cách điều hướng chỉ động chạm đến Router tương ứng, không ảnh hưởng luồng nghiệp vụ.

- **Cô lập và dễ kiểm thử (testability)**: Tách biệt phần điều hướng UI vào router giúp việc kiểm thử unit cho coordinator trở nên khả thi và nhẹ nhàng hơn. Ta có thể mock một **Router** (ví dụ tạo một DummyRouter tuân thủ protocol) để kiểm tra xem Coordinator gọi đúng phương thức điều hướng hay không, **mà không cần phụ thuộc vào UINavigationController thật**. Nhờ đó, dòng điều hướng được test độc lập với UI framework. Benoit Pasquier – một iOS developer – nhận xét rằng ban đầu việc thêm Router có vẻ dư thừa, nhưng thực tế nó cải thiện rõ rệt **tính maintainability và testability**: ta có thể **unit test coordinator bằng cách giả lập router** và dễ dàng kiểm tra các nhánh điều hướng khác nhau mà không cần dựng cả stack UI ⁷. Nếu không có router, việc test code coordinator phức tạp hơn nhiều (vì phải tương tác với UIViewController hoặc UINavigationController thực).
- **Phân tách rõ ràng vai trò, tránh trùng lặp code**: Router chịu trách nhiệm “**hiện thực**” việc chuyển màn hình, ví dụ: đẩy view controller vào navigation stack, trình bày màn hình modally, hoặc thậm chí xử lý callback khi màn hình biến mất. Trong khi đó, Coordinator chỉ lo “**quyết định**” màn hình tiếp theo là gì. Nhờ đó, code điều hướng UI tập trung hết trong router thay vì rải rác ở nhiều coordinator. Các Coordinator khác nhau có thể dùng chung một router (ví dụ cùng đẩy lên một navigation chính), tránh lặp lại mã điều khiển UIKit. Đồng thời, do Router thường được thiết kế không giữ **tham chiếu ngược** đến Coordinator, nên nó không tạo cycle, giữ cho kiến trúc gọn gàng. Router cũng có thể tận dụng các delegate của UIKit (như UINavigationControllerDelegate) để thông báo sự kiện điều hướng (như đã pop xong) – qua đó ta có thể trigger coordinator thu hồi child khi người dùng bấm nút Back, tránh memory leak. Tất cả những điều này góp phần làm ứng dụng dễ **bảo trì** hơn, vì mỗi thay đổi trong logic điều hướng (UI) hay logic luồng (flow) đều ở module tách biệt.

Ví dụ minh họa: Giả sử ứng dụng có luồng đăng nhập. `AppCoordinator` khi khởi động sẽ kiểm tra tình trạng xác thực, nếu chưa đăng nhập thì khởi tạo `AuthCoordinator` (một child coordinator) để quản lý luồng đăng nhập. `AuthCoordinator` tuân thủ `Coordinator` protocol, nên có phương thức `start()` để trình bày màn hình đăng nhập đầu tiên. Tuy nhiên, thay vì tự nó `present LoginViewController`, `AuthCoordinator` sẽ gọi `router.present(loginVC, animated: true)` thông qua một đối tượng `router` đã được truyền vào (ví dụ `router` này gói bên trong một `UINavigationController`). Router sẽ lo việc `pushViewController` tương ứng trên navigation controller thật. Khi người dùng hoàn thành đăng nhập, `AuthCoordinator` có thể gọi callback `onFinished` để báo cho `AppCoordinator` biết flow đã xong, qua đó `AppCoordinator` **remove** `AuthCoordinator` khỏi danh sách child và chuyển sang luồng chính (ví dụ khởi chạy `MainCoordinator`). Trong suốt quá trình này, `AuthCoordinator` không hề gọi trực tiếp bất kỳ API UIKit nào để chuyển màn hình – mọi thao tác như push/pop do Router đảm nhiệm. Điều này cho phép `AuthCoordinator` dễ dàng được thay thế logic điều hướng (ví dụ chuyển sang dùng modal thay vì push) chỉ bằng cách cung cấp một Router khác, mà không cần sửa code điều phối luồng bên trong nó.

Tổng kết lại, `Router` protocol được thiết kế nhằm **tách biệt phần “chuyển màn hình” khỏi phần “điều phối luồng”**. Nó giải quyết vấn đề phụ thuộc UI của Coordinator và tăng cường tính module hóa cho kiến trúc. Sự kết hợp của `Coordinator` protocol và `Router` protocol trong Coordinator Pattern mang lại một cấu trúc ứng dụng rõ ràng: mỗi **Coordinator** quản lý một luồng, tuân thủ hợp đồng chung (quản lý children, vòng đời, v.v.), và sử dụng một **Router** để thực hiện thao tác điều hướng UI. Cách tiếp cận này

đảm bảo rằng logic điều hướng được centralize nhưng vẫn linh hoạt, **dễ mở rộng, dễ kiểm thử và dễ bảo trì** trong các dự án iOS quy mô chuyên nghiệp ⁴ ⁷ .

Nguồn tham khảo: Các tài liệu về *Coordinator Pattern* cho thấy việc dùng protocol giúp tách rời các thành phần trong kiến trúc. Ray Wenderlich mô tả rằng coordinator protocol định nghĩa những thuộc tính như `children` và `router` để mọi coordinator cụ thể tuân theo, qua đó **decouple quan hệ cha-con** giữa các flow ¹ . Tương tự, router protocol định nghĩa các hàm điều hướng (`present`, `dismiss`...), tách coordinator khỏi chi tiết trình bày view controller ⁵ . Chính nhờ cách tiếp cận này, view controllers không cần biết nhau mà vẫn liên kết được thông qua coordinator, và khi muốn mở rộng ứng dụng với flow mới, ta chỉ việc tạo thêm coordinator mới mà không đụng chạm phần còn lại ² . Những lợi ích về **Single Responsibility, reusability, và testability** của mô hình này đã được nhiều kỹ sư kiểm chứng – ví dụ, Benoit Pasquier nhấn mạnh khả năng unit test coordinator bằng cách mock router, giúp kiểm thử các nhánh điều hướng mà không phụ thuộc UI thật ⁷ . Tổng thể, `Coordinator` và `Router` protocol là hai trụ cột giúp hiện thực hoá kiến trúc điều hướng linh hoạt, rõ ràng và vững chắc cho ứng dụng iOS hiện đại.

Sources:

1. Chetan Kumar, “iOS Coordinator Pattern” (2023) – Giới thiệu cách tách logic routing ra khỏi ViewController, nhấn mạnh tính rõ ràng, maintainability ⁸ ⁴ .
2. Kodeco (Ray Wenderlich), “Coordinator Pattern” – Định nghĩa vai trò của Coordinator và Router protocol, cách chúng decouple các thành phần trong flow điều hướng ¹ ⁵ ² .
3. Benoit Pasquier, “Coordinator & MVVM – Clean Navigation” (2019) – Thảo luận việc bổ sung Router để bắt sự kiện back button, nâng cao testability, tránh memory leak; khẳng định lợi ích của việc mock Router khi test Coordinator ⁷ .
4. Hoàng Nguyễn, “Swift – Coordinator Pattern” (2019) – Bài viết tiếng Việt giải thích Coordinator là đối tượng quản lý navigation flow, tách biệt khỏi business logic ³ .

¹ ² ⁵ ⁶ Design Patterns by Tutorials, Chapter 23: Coordinator Pattern | Kodeco
<https://www.kodeco.com/books/design-patterns-by-tutorials/v3.0/chapters/23-coordinator-pattern>

³ Swift - Coordinator Pattern
<https://viblo.asia/p/swift-coordinator-pattern-gDVK2RLrKLj>

⁴ ⁸ iOS Coordinator Pattern. In todays article, let's explore... | by Chetan Kumar | Medium
https://medium.com/@chetankumar_17171/ios-coordinator-pattern-504d9361d77b

⁷ Coordinator & MVVM - Clean Navigation and Back Button in Swift
<https://benoitpasquier.com/coordinator-pattern-navigation-back-button-swift/>